This indicates that the model correctly classified all test samples.

Insights

Logistic regression provides valuable insights and robust performance for classification tasks. Key points to consider:

- **Probability Scores:** Logistic regression outputs probabilities, which can be interpreted as confidence levels for predictions.
- **Threshold Tuning:** The default threshold of 0.5 can be adjusted to optimize performance for specific applications.
- **Feature Importance:** The coefficients of the logistic regression model indicate the importance of each feature in predicting the target variable.

Advanced Considerations

Handling Imbalanced Datasets

For datasets where one class significantly outnumbers the other, accuracy may not fully represent model performance. Use metrics like precision, recall, and F1-score instead: from sklearn.metrics import precision_score, recall_score, f1_score precision = precision_score(y_test, predictions) recall = recall_score(y_test, predictions)

f1 = f1_score(y_test, predictions)
print("Precision:", precision)
print("Recall:", recall)
print("F1-Score:", f1)

Multiclass Classification

Logistic regression can handle multiclass problems using the multi_class parameter in Scikit-Learn: log_model = LogisticRegression(multi_class='multinomial', solver='lbfgs') log_model.fit(X_train, y_train)

Regularization

Regularization techniques like L1 and L2 can prevent overfitting by penalizing large coefficients: log_model = LogisticRegression(penalty='l2', C=1.0) log_model.fit(X_train, y_train)